



PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

100-ft Ring Road, Bengaluru – 560 085, Karnataka, India

Design and Implementation of Power-Efficient BIST using Bit-Swapping LFSR and MISR

TEAM MEMBERS:

SL.No	NAME	SRN
1]	Divyashree Chakravarthi	PES1UG23EC904
2]	Madhumitha P	PES2UG23EC080

Under the Guidance of

Dr. Nirmala Devi M

Professor, Department of Electronics and Communication Engineering

Faculty of Engineering, PES University

B.Tech Program

1. Abstract

With increasing complexity in VLSI systems, testing becomes a major challenge due to high cost, power consumption, and time. Built-In Self-Test (BIST) provides an efficient solution by enabling on-chip testing.

This project proposes a power-efficient BIST architecture using:

- Bit-Swapping Linear Feedback Shift Register (LFSR) for reduced switching activity
- Modified Multiple Input Signature Register (MISR) for simplified response analysis
- A 4:1 Multiplexer and an 8:1 Multiplexer as Circuit Under Test (CUT)

The design was first implemented and validated on an Artix-7 FPGA using Vivado, followed by an optimised simulation-based architecture with CUT integration, initially implemented using 4:1 MUX as CUT, and later on extended the application for an 8:1 MUX to ensure the design is flexible.

2. Introduction

Modern VLSI systems contain millions of transistors, increasing the probability of faults. Traditional external testing:

- Requires expensive Automatic Test Equipment (ATE)
- Consumes high power
- Is time-intensive

Built-In Self-Test (BIST) overcomes these issues by embedding test generation and response analysis within the chip.

However, conventional BIST suffers from:

- High switching activity in LFSR → increased dynamic power
 - Complex MISR analysis for multi-output circuits
-

3. Proposed Methodology

3.1 Overall BIST Architecture

The system consists of:

1. Test Pattern Generator (TPG) → Bit-Swapping LFSR
 2. Circuit Under Test (CUT) → 4:1 MUX , 8:1 MUX
 3. Output Response Analyzer (ORA) → Modified MISR
 4. Comparator → Pass/Fail detection
-

4. Bit-Swapping LFSR

Concept

Conventional LFSR generates pseudo-random patterns but causes high switching transitions.

To reduce this:

- Conditional swapping is introduced between adjacent bits
- Controlled by MSB

Operation:

- If MSB = 0 → swap bits
- If MSB = 1 → no swap

This reduces toggling → lowers dynamic power.

Verilog Design

```
module bs_lfsr_top (  
    input clk,  
    input rst,  
    output [3:0] led  
);  
  
    reg [25:0] counter = 0;  
    wire slow_clk;  
  
    always @(posedge clk or posedge rst) begin  
        if (rst)  
            counter <= 0;  
        else  
            counter <= counter + 1;  
    end  
  
    assign slow_clk = counter[3];  
  
    reg [3:0] lfsr = 4'b1000;  
    wire feedback;  
    wire [3:0] next_lfsr;  
    wire [3:0] swapped;  
  
    assign feedback = lfsr[3] ^ lfsr[0];  
    assign next_lfsr = {feedback, lfsr[3:1]};  
  
    assign swapped[0] = (lfsr[3] == 0) ? next_lfsr[1] : next_lfsr[0];  
    assign swapped[1] = (lfsr[3] == 0) ? next_lfsr[0] : next_lfsr[1];  
    assign swapped[3:2] = next_lfsr[3:2];  
  
    always @(posedge slow_clk or posedge rst) begin
```

```
if (rst)
    lfsr <= 4'b1000;
else
    lfsr <= swapped;
end
```

```
assign led = lfsr;
```

```
endmodule
```

Testbench

```
module tb_bs_lfsr;
reg clk;
reg rst;
wire [3:0] led;
```

```
bs_lfsr_top uut (
    .clk(clk),
    .rst(rst),
    .led(led)
);
```

```
always #5 clk = ~clk;
```

```
initial begin
    clk = 0;
    rst = 1;
    #20 rst = 0;
    #5000;
    $finish;
end
```

```
initial begin
    $monitor("Time=%0t | LFSR Output=%b", $time, led);
end
endmodule
```

5. Modified MISR

Concept

Instead of handling multiple outputs separately:

- Outputs are XOR-combined into a single stream
 - Simplifies signature analysis
-

Verilog Code

```
module misr(
    input clk,
    input rst,
    input [3:0] data_in,
    output reg [3:0] signature
);

    wire feedback;
    assign feedback = signature[3] ^ data_in[0];

    always @(posedge clk or posedge rst) begin
        if (rst)
            signature <= 4'b0000;
        else begin
            signature[3] <= feedback;
        end
    end
endmodule
```



```
        signature[2] <= signature[3] ^ data_in[1];
        signature[1] <= signature[2] ^ data_in[2];
        signature[0] <= signature[1] ^ data_in[3];
    end
end

endmodule
```

Testbench

```
module tb_misr;
reg clk;
reg rst;
reg [3:0] data_in;
wire [3:0] signature;

misr uut (
    .clk(clk),
    .rst(rst),
    .data_in(data_in),
    .signature(signature)
);

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    data_in = 4'b0000;
    #20 rst = 0;

    #10 data_in = 4'b1010;
    #10 data_in = 4'b1100;
```

```
#10 data_in = 4'b0111;  
#10 data_in = 4'b1111;  
#10 data_in = 4'b0011;  
#10 data_in = 4'b1001;
```

```
#50 $finish;  
end
```

```
endmodule
```

6. Circuit Under Test (CUT) – 4:1 MUX

```
module mux4to1_cut(  
input i0, i1, i2, i3,  
input s0, s1,  
output y  
);
```

```
wire s0_bar, s1_bar;  
wire w0, w1, w2, w3;
```

```
not (s0_bar, s0);  
not (s1_bar, s1);
```

```
and (w0, i0, s1_bar, s0_bar);  
and (w1, i1, s1_bar, s0);  
and (w2, i2, s1, s0_bar);  
and (w3, i3, s1, s0);
```

```
or (y, w0, w1, w2, w3);
```

```
Endmodule
```


In this project, bit swapping of the LFSR was implemented using two Circuits Under Test (CUTs). Initially, a **4:1 multiplexer** was employed as the CUT. Subsequently, the design was optimized to handle increased complexity by utilizing an **8:1 multiplexer**, thereby improving the circuit's functionality and efficiency.

7. Complete BIST Integration

```
module bist_mux_top(  
    input clk,  
    input rst,  
    output [3:0] test_pattern,  
    output cut_out,  
    output [3:0] signature,  
    output pass_fail  
);
```

```
    wire [3:0] led;
```

```
    bs_lfsr_top prpg (  
        .clk(clk),  
        .rst(rst),  
        .led(led)  
    );
```

```
    assign test_pattern = led;
```

```
// CUT
```

```
module mux8to1_cut(  
    input i0,  
    input i1,  
    input i2,
```

```

    input i3,
    input i4,
    input i5,
    input i6,
    input i7,
    input s0,
    input s1,
    input s2,
    output y
);
    wire s0_bar, s1_bar, s2_bar;
wire w0, w1, w2, w3, w4, w5, w6, w7;

// Inverters
not (s0_bar, s0);
not (s1_bar, s1);
not (s2_bar, s2);

// AND gates for each input path
and (w0, i0, s2_bar, s1_bar, s0_bar); // 000
and (w1, i1, s2_bar, s1_bar, s0);    // 001
and (w2, i2, s2_bar, s1,    s0_bar); // 010
and (w3, i3, s2_bar, s1,    s0);    // 011
and (w4, i4, s2,    s1_bar, s0_bar); // 100
and (w5, i5, s2,    s1_bar, s0);    // 101
and (w6, i6, s2,    s1,    s0_bar); // 110
and (w7, i7, s2,    s1,    s0);    // 111

// OR gate
or (y, w0, w1, w2, w3, w4, w5, w6, w7);
endmodule

```

```
// MISR
wire [3:0] misr_in;
assign misr_in = {3'b000, mux_y};

misr resp (
    .clk(clk),
    .rst(rst),
    .data_in(misr_in),
    .signature(signature)
);

// Comparator
assign pass_fail = (signature == 4'b0100);

Endmodule
```

FULL CODE:

```
module bs_lfsr8(
    input clk,
    input rst,
    output [7:0] led
);
    reg [7:0] lfsr;
wire feedback;
wire [7:0] next_lfsr;
wire [7:0] swapped;

assign feedback = lfsr[7] ^ lfsr[5];
assign next_lfsr = {feedback, lfsr[7:1]};

// swap lower 2 bits when MSB = 0
```

```
assign swapped[0] = (lfsr[7]==0) ? next_lfsr[1] : next_lfsr[0];
assign swapped[1] = (lfsr[7]==0) ? next_lfsr[0] : next_lfsr[1];
assign swapped[7:2] = next_lfsr[7:2];
```

```
always @(posedge clk or posedge rst)
```

```
begin
```

```
    if(rst)
```

```
        lfsr <= 8'b10000000;
```

```
    else
```

```
        lfsr <= swapped;
```

```
end
```

```
assign led = lfsr;
```

```
endmodule
```

```
// 8:1 MUX using basic gates
```

```
module mux8to1_cut_new(
    input i0,input i1,input i2,input i3,
    input i4,input i5,input i6,input i7,
    input s0,input s1,input s2,
    output y
);
```

```
wire s0b,s1b,s2b;
```

```
wire w0,w1,w2,w3,w4,w5,w6,w7;
```

```
not(s0b,s0);
```

```
not(s1b,s1);
```

```
not(s2b,s2);
```

```
and(w0,i0,s2b,s1b,s0b);
and(w1,i1,s2b,s1b,s0 );
and(w2,i2,s2b,s1 ,s0b);

and(w3,i3,s2b,s1 ,s0 );
and(w4,i4,s2 ,s1b,s0b);
and(w5,i5,s2 ,s1b,s0 );
and(w6,i6,s2 ,s1 ,s0b);
and(w7,i7,s2 ,s1 ,s0 );

or(y,w0,w1,w2,w3,w4,w5,w6,w7);

endmodule
```

```
// 8-bit MISR
module misr8(
    input clk,
    input rst,
    input [7:0] data_in,
    output reg [7:0] signature
);

wire feedback;

assign feedback = signature[7] ^ data_in[0];

always @(posedge clk or posedge rst)
begin
    if(rst)
        signature <= 8'b00000000;
```

```
    else begin
        signature[7] <= feedback;
        signature[6] <= signature[7] ^ data_in[1];
        signature[5] <= signature[6] ^ data_in[2];
        signature[4] <= signature[5] ^ data_in[3];
        signature[3] <= signature[4] ^ data_in[4];
        signature[2] <= signature[3] ^ data_in[5];
        signature[1] <= signature[2] ^ data_in[6];
        signature[0] <= signature[1] ^ data_in[7];
    end
end

endmodule
```

```
// TOP MODULE : BIST for 8:1 MUX
```

```
module bist_mux8_top(
    input clk,
    input rst,
    output [7:0] test_pattern,
    output cut_out,
    output [7:0] signature,
    output pass_fail
);
```

```
    wire [7:0] led;
    wire mux_y;
    wire [7:0] misr_in;
```

```
// PRPG
```



```
bs_lfsr8 prpg(  
    .clk(clk),  
    .rst(rst),  
    .led(led));  
assign test_pattern = led;
```

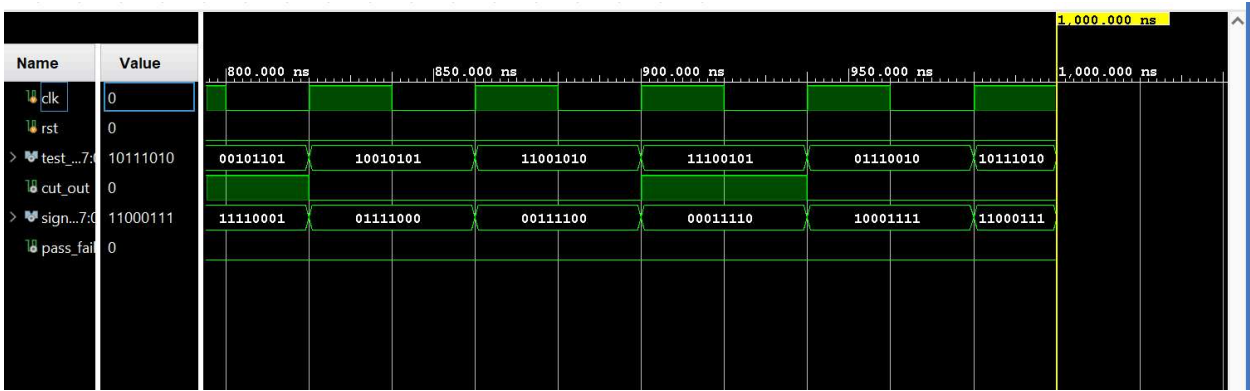
```
// CUT  
mux8to1_cut cut(  
    .i0(led[0]),  
    .i1(led[1]),  
    .i2(led[2]),  
    .i3(led[3]),  
    .i4(led[4]),  
    .i5(led[5]),  
    .i6(led[6]),  
    .i7(led[7]),  
    .s0(led[0]),  
    .s1(led[1]),  
    .s2(led[2]),  
    .y(mux_y)  
);
```

```
assign cut_out = mux_y;
```

```
// MISR  
assign misr_in = {7'b00000000,mux_y};
```

```
misr8 resp(  
    .clk(clk),  
    .rst(rst),  
    .data_in(misr_in),  
    .signature(signature)
```

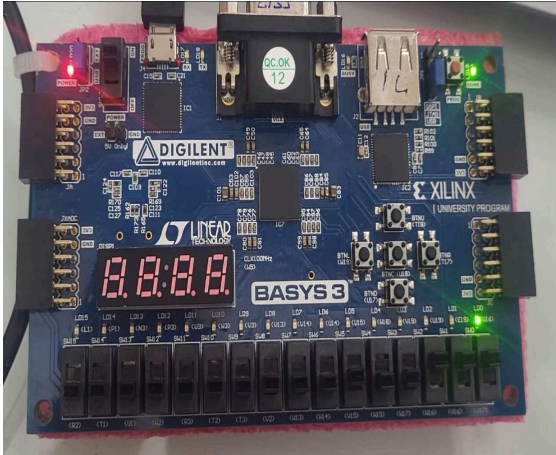

OVERALL BIST:



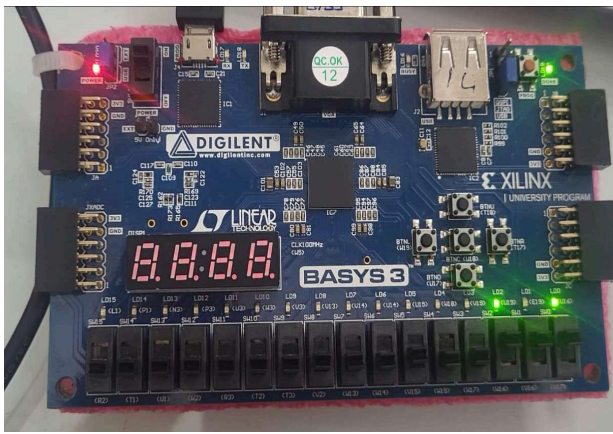
9. FPGA Implementation (Artix-7)

Prototype Implementation

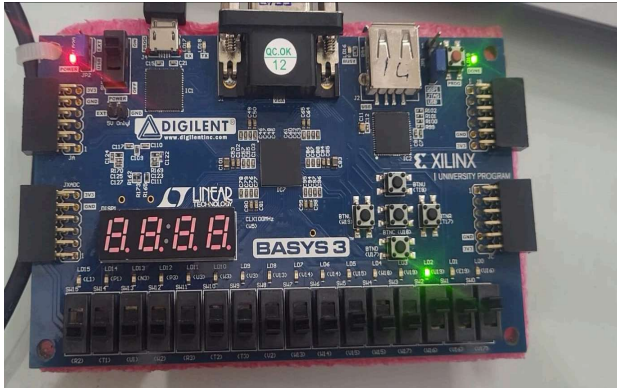
- Implemented using Vivado
- Bitstream generated
- Outputs mapped to LEDs



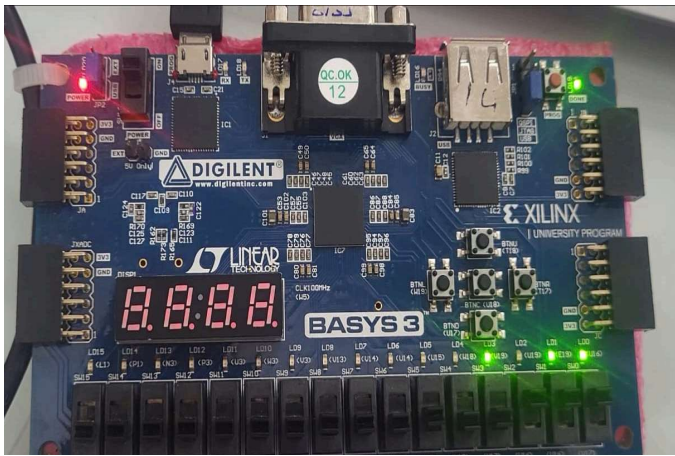
pattern: 0001



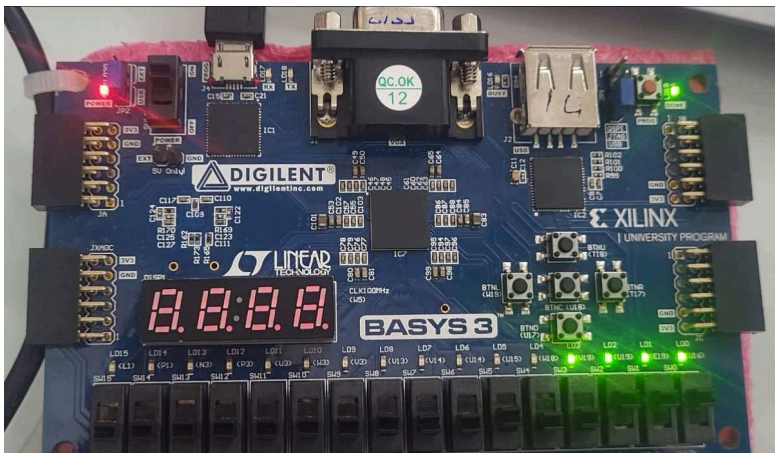
pattern: 0101



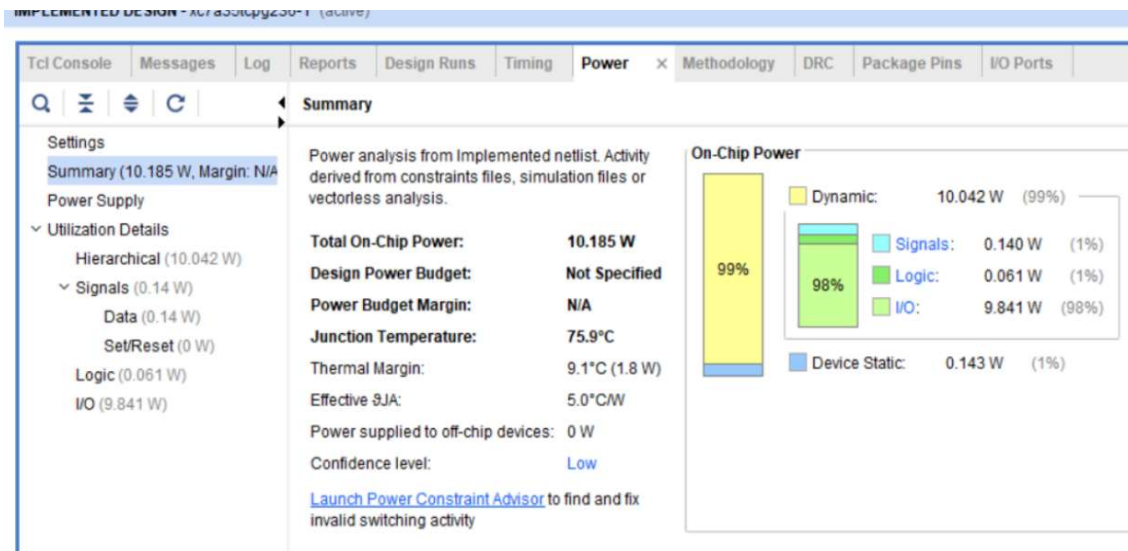
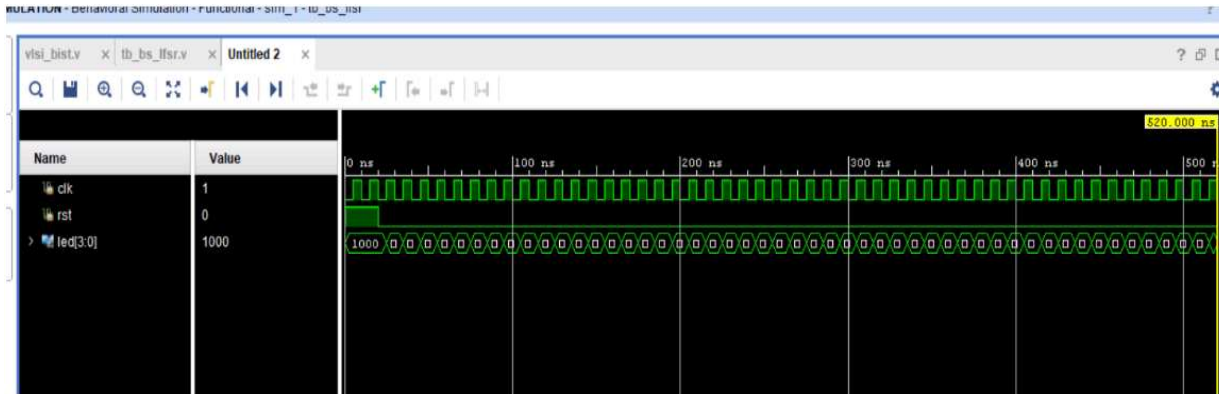
Pattern:0100



pattern:1011



pattern: 1111



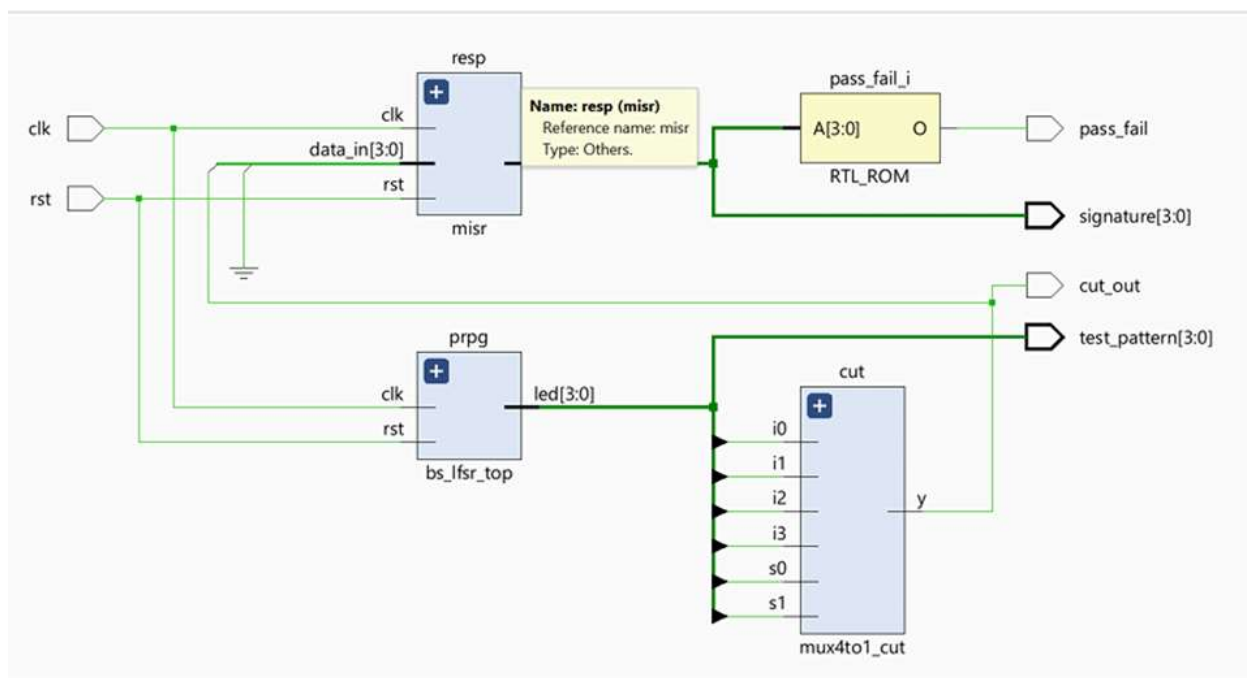
Observation:

- Waveform verification successful
- On-chip power ≈ 10.18 W (prototype)
- Resource usage includes LUTs, FFs, BUFG, etc.

Timing Analysis

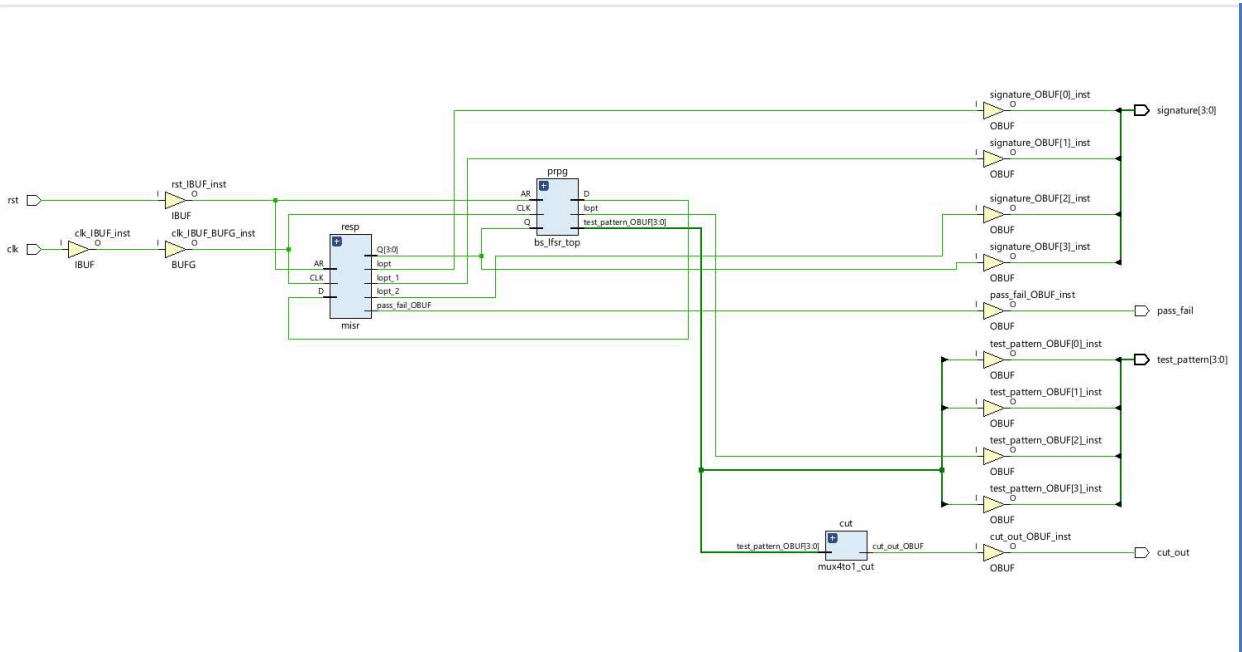
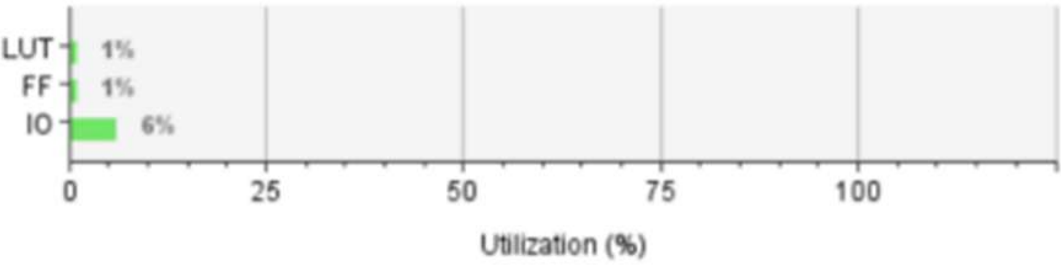
- Some constraints are missing (as shown in timing report page 2)
→ This is expected in the prototype stage

9. Synthesized Design Results



Summary

Resource	Utilization	Available	Utilization %
LUT	3	20800	0.01
FF	31	41600	0.07
IO	6	106	5.66



From final integrated design:

- LUTs used: 6
- Registers: 12
- Power: ~6.26 W

Shows power reduction compared to the prototype

10. Results and Discussion

- Bit-Swapping LFSR:
 - Reduced switching activity
 - Maintained randomness
 - MISR:
 - Successfully compressed output
 - Simplified signature analysis
 - BIST System:
 - Correct PASS/FAIL detection
 - Functional verification via simulation
-

11. Novelty of the Project

Existing

- LFSR + MISR-based BIST (standard)

Our Contribution

- Conditional bit-swapping based on MSB
 - Reduced switching transitions → lower dynamic power
 - Simplified MISR using output aggregation
 - FPGA validation + power comparison
-

12. Future Scope

Technical Extensions

- Use higher-order LFSR (8-bit / 16-bit)
- Test complex CUTs (ALU, memory, DSP blocks)
- Implement scan-based BIST
- Integrate the fault injection mechanism
- Use low-power clock gating

Advanced Research Scope

- AI-based test pattern optimization
 - Adaptive LFSR (dynamic polynomial selection)
 - Hardware Trojan detection using BIST
 - ASIC-level implementation
-

13. Conclusion

A power-efficient BIST architecture was successfully designed and implemented.

The Bit-Swapping LFSR reduced switching activity, leading to lower power consumption, while the modified MISR simplified response analysis.

FPGA implementation and simulation results validate the effectiveness of the proposed system.

14. References

- [1] S. Guru Sharan, S. Jeeshnu, P. Harish Annamalai, S. Haroon Rasheed and E. Prabhu, "Design and Implementation of a Power Efficient BIST," 2021 5th International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 2021, pp. 555-561, doi: 10.1109/ICCMC51019.2021.9418462.
- [2]K. N. Devika and R. Bhakthavatchalu, "Design of efficient programmable test-per-scan logic BIST modules," 2017 International conference on Microelectronic Devices, Circuits and Systems (ICMDCS), Vellore, India, 2017, pp. 1-6, doi: 10.1109/ICMDCS.2017.8211609.
- [3]Ch. E. Stroud, "A Designer's Guide to Built-In Self Test", Kluwer Academic Pubs., ISBN 1-4020-7050-0, 2002.
- [4]J. K. Bhandari, M. K. Chaitanya and G. V. Rao, "A Low Power Test Pattern Generator for Minimizing Switching Activities and Power Consumption," 2018 International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, 2018, pp. 76-80, doi: 10.1109/ICIRCA.2018.8597212.
- [5]K. N. Devika and R. Bhakthavatchalu, "Programmable MISR modules for logic BIST based VLSI testing," 2016 International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT), Kumaracoil, 2016, pp. 699-703, doi: 10.1109/ICCICCT.2016.7988042